

CrossGL: A Hub-and-Spoke Compiler Infrastructure for Universal Shader Translation Across Heterogeneous GPU Architectures

Nripesh Niketan*, Vaatsalya Shrivastava, Bhushan Srivastav, and Sushant Mishra

CrossGL, United States
{nripesh, vaatsalya, bhushan, sushant}@crossgl.net

Abstract. The modern GPU programming landscape is fragmented across incompatible shading languages: GLSL for OpenGL, HLSL for DirectX, Metal Shading Language for Apple platforms, SPIR-V for Vulkan, and CUDA and HIP for general-purpose GPU computing. Developers targeting multiple platforms must therefore maintain separate shader codebases or depend on translation tools that cover only part of the language space.

We present CrossGL, a compiler infrastructure based on a hub-and-spoke architecture centered on a purpose-built intermediate shader language. By routing translation through a universal abstract syntax tree derived from CrossGL, the system reduces the number of required converters for N languages from $O(N^2)$ pairwise translators to $O(N)$ frontends and backends. CrossGL captures shader semantics across 16 GPU pipeline stages, including ray tracing and mesh stages, through a type system with vectors, matrices, textures, generics, pattern matching, and platform-agnostic semantic annotations.

We evaluate a reference implementation with nine forward code generators and native import frontends. On an eight-program example corpus covering graphics, compute, arrays, generic pattern matching, and a 489-line physically based rendering shader, CrossGL successfully generated 56 of 60 compatible target outputs, with a median translation time of 30.83 ms per translation on an Apple M2 Max. A 767-test unit suite validates the lexer, parser, backend contracts, and native import paths. The results show that the hub-and-spoke design is practical while also identifying the remaining gaps in SPIR-V validation and advanced backend completeness.

Keywords: shader translation, cross-platform GPU computing, intermediate representation, compiler infrastructure, heterogeneous architectures

* ORCID: 0009-0008-2066-1937

1 Introduction

The graphics processing unit (GPU) ecosystem has become increasingly fragmented as hardware vendors and platform owners have developed competing shading and compute languages. OpenGL applications use GLSL [1], Microsoft’s DirectX ecosystem relies on HLSL [2], Apple’s platforms require Metal Shading Language [3], and Vulkan defines SPIR-V as its portable binary intermediate representation [4]. Beyond graphics, general-purpose GPU (GPGPU) computing adds further fragmentation: NVIDIA’s CUDA [5] dominates the market while AMD’s HIP [6] provides an alternative, and emerging languages such as Mojo [7] and Slang [8] introduce yet more options. Developers who wish to target multiple platforms must either maintain separate codebases for each language or employ ad hoc translation scripts, leading to duplicated engineering effort, inconsistent behavior across platforms, and significant portability barriers.

Several tools attempt to address this fragmentation. SPIRV-Cross [9] decompiles SPIR-V binaries to GLSL, HLSL, and Metal, but operates only in one direction and cannot target CUDA, HIP, Rust, or Mojo. Google’s Tint [10] serves the WebGPU ecosystem by compiling WGSL but is not designed as a general-purpose translator. Mozilla’s Naga [11] supports translation among WGSL, GLSL, SPIR-V, and Metal within the `wgpu` project but covers a limited language set. Microsoft’s DirectX Shader Compiler (DXC) [12] compiles HLSL to DXIL and SPIR-V but remains HLSL-centric. Crucially, each of these tools implements pairwise converters: supporting N languages with direct translation requires up to $N(N - 1)$ converters, a quadratic scaling that makes comprehensive cross-platform support impractical.

We introduce CrossGL, a compiler infrastructure that addresses this challenge through a *hub-and-spoke* translation model. At the center of the architecture sits the CrossGL language, a purpose-built intermediate shader language with a rich type system, explicit pipeline stage semantics, and support for modern GPU features. Any source language is first translated to CrossGL through a backend-specific parser and reverse code generator; the resulting CrossGL representation is then translated to any target language through a corresponding forward code generator. This design requires only $2N$ converters for N languages— N reverse converters (source to CrossGL) and N forward converters (CrossGL to target)—reducing the quadratic scaling to linear.

The CrossGL language was designed as a semantic superset of shader concepts across all supported platforms. It provides explicit syntax for 16 GPU pipeline stages spanning traditional graphics (vertex, fragment, geometry, tessellation), modern mesh and task shaders, and all six DirectX Raytracing (DXR) stages. Its type system encompasses sized integer and floating-point primitives, vector and matrix types, texture and sampler abstractions, generics with trait constraints, and Rust-inspired pattern matching. A dual abstract syntax tree (AST) architecture separates backend-specific representations, which faithfully capture source language semantics, from a universal AST that serves as the normalized form for code generation.

This paper makes the following contributions:

- The CrossGL intermediate shader language, a human-readable IR designed to capture GPU shader semantics across 16 pipeline stages, generics, pattern matching, and platform-agnostic semantic annotations.
- A hub-and-spoke translation architecture that reduces the complexity of cross-platform shader translation from $O(N^2)$ pairwise converters to $O(N)$, where each new language requires only two converters regardless of how many languages are already supported.
- A dual-AST design that cleanly separates backend-specific language representations from a universal normalized form, enabling faithful round-trip translation.
- An extensible plugin architecture that enables addition of new backends without modification to the core translation engine, as demonstrated through prototype backends spanning graphics shaders, GPU compute, and general-purpose languages.
- An open-source reference implementation demonstrating the feasibility of the approach across diverse target languages including GLSL, HLSL, Metal, SPIR-V, CUDA, HIP, Rust, Mojo, and Slang.

The remainder of this paper is organized as follows. Section 2 surveys background and related work. Section 3 describes the CrossGL language design. Section 4 presents the system architecture. Section 5 details the code generation strategies. Section 6 evaluates the system. Section 7 discusses limitations and future directions. Section 8 concludes.

2 Background and Related Work

2.1 GPU Programming Landscape

The proliferation of GPU hardware across desktop, mobile, and cloud platforms has produced a diverse ecosystem of shading and compute languages, each tailored to a specific graphics API or vendor architecture. Table 1 summarizes the principal languages and their characteristics.

GLSL [1] is the standard shading language for OpenGL and can also be used with Vulkan through compilation to SPIR-V. HLSL [2] powers the DirectX ecosystem and has recently gained SPIR-V support through Microsoft’s DXC compiler [12]. Apple’s Metal Shading Language [3] is required for GPU programming on iOS and macOS. On the compute side, CUDA [5] remains the dominant language for NVIDIA hardware, while AMD’s HIP [6] provides source-level compatibility with CUDA for AMD GPUs. Emerging languages include Slang [8], which introduces extensible shader programming abstractions, and Mojo [7], which targets high-performance computing and machine learning workloads.

These languages share many fundamental concepts—vectors, matrices, textures, samplers, control flow—but differ in syntax, type naming conventions, pipeline stage models, and semantic annotation mechanisms [14]. This overlap motivates an intermediate representation that can capture their shared semantics while preserving platform-specific distinctions.

Table 1. Overview of GPU shading and compute languages

Language	Platform	Domain	Stages
GLSL [1]	OpenGL/Vulkan	Graphics/Compute	10
HLSL [2]	DirectX	Graphics/Compute	14
Metal [3]	Apple	Graphics/Compute	8
SPIR-V [4]	Vulkan	Binary IR	14
CUDA [5]	NVIDIA	Compute	1
HIP [6]	AMD/NVIDIA	Compute	1
Slang [8]	Cross-platform	Graphics/Compute	14
Mojo [7]	Cross-platform	Compute/ML	1
Rust GPU [13]	Cross-platform	Graphics/Compute	Varies

2.2 Existing Translation Tools

Several tools have been developed to bridge the gaps between GPU shading languages. Table 2 compares their public translation scope. The table is intentionally conservative: it distinguishes tools that accept one primary source language from tools that expose multiple source frontends, and it separates graphics-oriented outputs from compute-oriented outputs.

Table 2. Comparison of existing shader translation tools

Tool	Input model	Main outputs	Beyond graphics	Native bidir.
SPIRV-Cross [9]	SPIR-V only	GLSL, HLSL, MSL	None	No
Tint/Dawn [10]	WGSL; optional SPIR-V reader	WGSL, SPIR-V; WebGPU lowering	None	Partial
Naga [11]	WGSL, SPIR-V, GLSL	SPIR-V, WGSL, MSL, HLSL, GLSL	None	Partial
DXC [12]	HLSL	DXIL, SPIR-V; Metal with converter	None	No
Slang [8]	Slang/HLSL	HLSL, GLSL, SPIR-V, MSL, CUDA, C++	CUDA, C++	Partial
CrossGL	CrossGL plus native frontends	GLSL, HLSL, Metal, SPIR-V, CUDA, HIP, Rust, Mojo, Slang	CUDA, HIP, Rust, Mojo	Yes

SPIRV-Cross [9], maintained by the Khronos Group, decompiles SPIR-V binary modules into GLSL, HLSL, and Metal source code. While widely used in game engines and graphics frameworks, it operates only in one direction (from SPIR-V) and cannot target compute-oriented languages such as CUDA, HIP, or Mojo.

Tint [10] is Google’s shader compiler for the WebGPU standard. It compiles WGSL (WebGPU Shading Language) to SPIR-V, Metal, HLSL, and GLSL. However, it accepts only WGSL as input and serves primarily the web platform rather than native GPU development.

Naga [11] is the shader translation module within Mozilla’s **wgpu** project. It supports WGSL, GLSL, and SPIR-V as inputs and can emit WGSL, SPIR-V, Metal, HLSL, and GLSL. While more versatile than single-source tools, its language coverage excludes CUDA, HIP, Rust, Mojo, and Slang.

DXC [12] compiles HLSL to DXIL (DirectX Intermediate Language) and SPIR-V. It is HLSL-centric and does not support other source languages or non-DirectX/Vulkan targets.

Slang [8] introduces language-level mechanisms for extensible shading, compiling to HLSL, GLSL, and SPIR-V. It represents a new source language rather than a universal translator.

A common limitation across these tools is the pairwise translation model: each source-target pair requires a dedicated converter. For N languages, full coverage demands $N(N - 1)$ converters. CrossGL overcomes this through a hub-and-spoke architecture that centralizes translation through a single intermediate language.

2.3 Intermediate Representations for Shaders

Intermediate representations (IRs) have long been central to compiler design [15]. In the GPU domain, SPIR-V [4] serves as the standard binary IR for Vulkan, encoding shader programs in a structured binary format. While SPIR-V preserves rich type information and control flow structure, it is not human-readable, is designed primarily for Vulkan consumption, and does not naturally support bidirectional translation.

At a lower level, LLVM IR [16] provides a universal compilation target used by many languages, including CUDA (via NVPTX) and AMD GPU targets (via AMDGPU). However, LLVM IR operates at the instruction level and does not preserve high-level shader semantics such as pipeline stages, vertex attributes, or texture sampling operations, making it unsuitable as a shader-to-shader translation IR.

MLIR [17] offers a more extensible approach through its dialect system, which allows domain-specific abstractions to coexist within a common framework. While promising, MLIR does not yet include a comprehensive shader dialect that captures the full breadth of GPU pipeline stages and graphics-specific operations.

The Cg language [18], developed by NVIDIA, was an early attempt at a cross-platform shading language that could compile to both OpenGL and DirectX targets. While pioneering, Cg predated modern GPU features such as compute shaders, ray tracing, and mesh shaders, and has been deprecated since 2012.

The Vulkan Programming Guide [19] provides comprehensive documentation of the SPIR-V consumption model, and the GLSL specification [20] defines the

textual source language from which many CrossGL type and operator conventions derive.

CrossGL addresses this gap by providing a human-readable, high-level IR that preserves shader-specific semantics—pipeline stages, vertex semantics, texture operations, wave intrinsics, and ray tracing constructs—while remaining amenable to bidirectional translation across both graphics and compute languages. The following section describes this language in detail.

3 The CrossGL Language

CrossGL is a purpose-built intermediate shader language designed to serve as the universal hub in the translation architecture. This section describes its design goals, syntax and type system, and shader stage model.

3.1 Language Design Goals

The design of CrossGL was guided by five principles:

Universal expressiveness. The language must be capable of representing any construct expressible in any supported target language. This includes not only common shader operations (vector arithmetic, texture sampling, control flow) but also platform-specific features such as HLSL semantic annotations, Metal address space qualifiers, and CUDA thread indexing.

Semantic preservation. Pipeline stage information, vertex attribute semantics, compute workgroup dimensions, and other GPU-specific metadata must be first-class constructs in the language rather than annotations that could be lost during translation.

Human readability. Unlike binary IRs such as SPIR-V, CrossGL is designed to be readable and writable by developers. This facilitates debugging, manual inspection of translation results, and direct authoring of cross-platform shaders.

Extensibility. New GPU features—ray tracing stages, mesh shaders, wave operations—must be expressible without breaking backward compatibility with existing shaders.

Minimalism. The language includes only constructs that map to real GPU operations, avoiding unnecessary abstractions that would complicate translation fidelity.

3.2 Syntax and Type System

CrossGL uses a C-family syntax augmented with shader-specific constructs. Listing 1.1 shows a basic vertex-fragment shader.

```

1 shader SimpleShader {
2     struct VertexInput {
3         vec3 position;
4         vec2 texCoord;
5     }

```

```

6   struct VertexOutput {
7       vec2 uv;
8       vec4 position;
9   }
10  struct FragmentOutput {
11      vec4 color;
12  }
13  vertex {
14      VertexOutput main(VertexInput input) {
15          VertexOutput output;
16          output.uv = input.texCoord;
17          output.position = vec4(input.position, 1.0);
18          return output;
19      }
20  }
21  fragment {
22      FragmentOutput main(VertexOutput input) {
23          FragmentOutput output;
24          float r = input.uv.x;
25          float g = input.uv.y;
26          output.color = vec4(r, g, 0.5, 1.0);
27          return output;
28      }
29  }
30 }

```

Listing 1.1. A simple CrossGL vertex-fragment shader

The top-level construct is the **shader** block, which encapsulates all declarations and pipeline stage definitions for a single shader program. Within a shader block, developers define data structures using **struct** declarations and implement pipeline stages using named blocks (**vertex**, **fragment**, **compute**, etc.). Each stage contains a **main** entry point function and may include helper functions.

Type system. CrossGL provides a comprehensive type system spanning scalar, vector, matrix, and GPU-specific types. Table 3 summarizes the type categories.

Table 3. CrossGL type system categories

Category	Types
Signed integers	i8, i16, i32, i64, int
Unsigned integers	u8, u16, u32, u64, uint
Floating point	f16, f32, f64, half, float, double
Boolean	bool
Float vectors	vec2, vec3, vec4
Int/uint vectors	ivec2–ivec4, uvec2–uvec4
Bool vectors	bvec2, bvec3, bvec4
Matrices	mat2–mat4, mat2x3, mat4x2, etc.
Texture/sampler	sampler2D, samplerCube, sampler3D
Generics	generic<T>, generic<T: Numeric>

CrossGL also supports Rust-inspired features beyond the core shader vocabulary: **let** and **let mut** declarations, **match** expressions with pattern guards,

generic types with trait constraints, and `for-in` range iteration. These features, while uncommon in traditional shading languages, enable expressive cross-platform abstractions and facilitate translation from languages like Rust and Mojo that employ similar constructs.

Semantic annotations. CrossGL uses `@semantic` attributes to annotate shader input/output variables with platform-agnostic semantics. During code generation, these annotations map to each platform’s mechanism: GLSL locations or built-ins, HLSL `SV_` semantics, and Metal attribute annotations.

3.3 Shader Stage Model

CrossGL explicitly models 16 GPU pipeline stages, organized into four execution categories. Table 4 enumerates the supported stages.

Table 4. CrossGL pipeline stages by execution model

Execution Model	Stages
Traditional Graphics	<code>vertex</code> , <code>fragment</code> , <code>geometry</code> , <code>tessellation_control</code> , <code>tessellation_evaluation</code>
Compute	<code>compute</code>
Modern Graphics	<code>mesh</code> , <code>task</code> , <code>object</code>
Ray Tracing	<code>ray_generation</code> , <code>ray_intersection</code> , <code>ray_closest_hit</code> , <code>ray_miss</code> , <code>ray_any_hit</code> , <code>ray_callable</code>

Compute stages support `layout()` qualifiers for workgroup dimensions and `shared` memory declarations for inter-thread communication. Ray tracing and wave operations are represented as first-class intrinsic calls, then mapped to HLSL wave intrinsics, Metal SIMD group functions, or GLSL subgroup operations during code generation.

4 System Architecture

CrossGL employs a hub-and-spoke architecture with the CrossGL language at the center and backend-specific modules at the periphery. This section describes the translation model, the dual-AST design, the compilation pipeline, and the plugin system.

4.1 Hub-and-Spoke Translation Model

The fundamental design principle of CrossGL is that every translation routes through a single intermediate language. Instead of direct converters for every source-target pair, the system decomposes each translation into at most two steps: source language to CrossGL, then CrossGL to the target language.

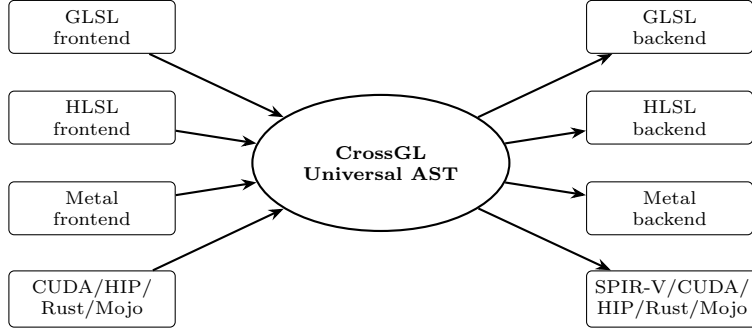


Fig. 1. Hub-and-spoke translation architecture. Each language contributes one import path to CrossGL and one export path from CrossGL, rather than direct converters to every other language.

This hub-and-spoke model supports three translation modes:

Forward translation converts CrossGL source code directly to a target language. The `.cgl` file is tokenized by the CrossGL lexer, parsed into a universal AST, and then emitted as target source code by a backend-specific code generator.

Reverse translation converts a native shader into CrossGL. The source file is tokenized and parsed by a backend-specific lexer and parser into a backend-specific AST, which is then converted to CrossGL source code by a reverse code generator.

Cross-translation combines the reverse and forward paths to translate between two non-CrossGL languages. For example, translating an HLSL shader to Metal proceeds as: HLSL $\rightarrow$ HLSL lexer/parser $\rightarrow$ HLSL AST $\rightarrow$ reverse codegen $\rightarrow$ CrossGL source $\rightarrow$ CrossGL lexer/parser $\rightarrow$ universal AST $\rightarrow$ Metal codegen $\rightarrow$ Metal source.

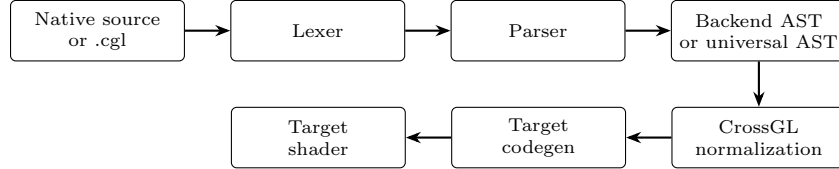


Fig. 2. Compilation pipeline for direct and cross-translation. Native inputs first pass through backend-specific parsers; CrossGL input enters directly at the universal AST path.

Complexity. For N supported languages, the hub-and-spoke model requires N forward code generators and N reverse converters, totaling $2N$ converters. A direct pairwise approach would require $N(N - 1)$ converters. With the current $N = 10$ languages, CrossGL uses 20 converters instead of 90, a $4.5\times$ reduction. Each additional language requires only two new converters rather than $2(N - 1)$.

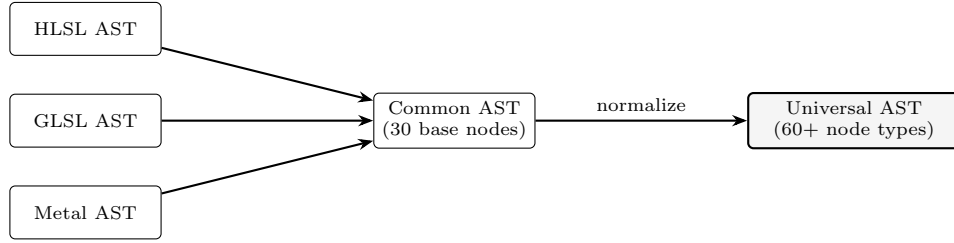


Fig. 3. Dual-AST architecture. Backend-specific ASTs preserve source language structure and extend a shared common AST. Normalization produces the universal AST used by all forward code generators.

4.2 Dual-AST Design

The system maintains two distinct AST layers, each optimized for a different purpose (Fig. 3). **The universal AST** is the canonical representation for forward code generation. It contains over 60 node types covering type nodes, shader program structure, declarations, statements, expressions, and GPU-specific operations such as texture, atomic, wave, ray tracing, mesh, and buffer operations. Every universal AST node supports visitor dispatch through `accept()` and carries an `annotations` dictionary for backend-specific metadata.

The backend ASTs provide native-language-oriented representations for reverse translation. Each backend parser preserves source-language structure before normalization—for example, HLSL constant buffers and semantic annotations remain explicit in the HLSL AST. The shared common AST supplies approximately 30 base node types that per-backend ASTs extend where the source language requires additional constructs.

The rationale for two AST layers is separation of concerns: backend parsers need to capture source language semantics without loss, while the forward code generators need a normalized, feature-rich representation that abstracts over platform differences. The universal AST serves as this normalized form.

4.3 Translation Pipeline

The forward translation pipeline consists of three stages: lexical analysis, parsing, and code generation.

Lexical analysis. The CrossGL lexer uses a combined regular expression pattern with named groups, compiled once at initialization. It recognizes over 220 token types spanning comments, preprocessor directives, keywords, type names, operators, and punctuation. A keyword post-lookup phase maps identifiers to their keyword token types. Token caching avoids redundant tuple allocation for frequently occurring tokens, and error reporting includes line and column numbers with surrounding context.

Parsing. The CrossGL parser is a hand-written recursive descent parser—a deliberate choice over parser generators to retain full control over error messages and recovery strategies. Expression parsing uses precedence climbing across 12

levels, from assignment (lowest) through ternary, logical, bitwise, comparison, shift, additive, multiplicative, and unary operators to postfix and primary expressions (highest). The parser employs speculative parsing with state save/restore for ambiguous constructs such as distinguishing function declarations from variable declarations. Dedicated recognition logic identifies GPU intrinsics during postfix expression parsing, including wave operations, ray tracing intrinsics, mesh shader operations, and ray query methods.

Code generation. Each target backend provides a code generator class that traverses the universal AST and emits target language source code. All code generators follow a common pattern: a top-level `generate()` method iterates over structs, globals, constant buffers, functions, and stages, then delegates to function, statement, and expression emitters. Type, semantic, and operator mappings are maintained as per-backend dictionaries.

Two registries coordinate the pipeline. The `SourceRegistry` maps file extensions to source specifications (lexer, parser, and optional reverse code generator factories), determining how input files are parsed. The `BackendRegistry` maps backend names and aliases to code generator classes, determining how output is generated.

4.4 Plugin Architecture

CrossGL supports dynamic backend discovery through a plugin loader. At initialization, the system scans backend subdirectories for `backend.spec.py` or `source.spec.py` modules, dynamically importing and registering them. This enables new backends to be added as drop-in packages without modifying core translation logic.

Each backend is described by a `BackendSpec` dataclass containing: a canonical name, a list of aliases (e.g., “hlsl” and “dx” for DirectX), supported file extensions, and a factory function for the code generator class. Both registries use lazy loading—backend modules are imported only when a translation targeting them is requested, keeping startup time minimal regardless of the number of installed backends.

5 Code Generation

This section details the type and semantic mapping mechanisms, the per-backend generation strategies, and the specialized SPIR-V assembly generator.

5.1 Type and Semantic Mapping

Each code generator maintains three lookup dictionaries that map CrossGL constructs to their target language equivalents: a type mapping, a semantic mapping, and a function mapping.

Type mapping. Table 5 illustrates how representative CrossGL types map across five major backends. CrossGL’s type names align with GLSL conventions,

so GLSL mappings are largely identity transformations. Other backends require systematic renaming: for example, HLSL and Metal use the `floatN` convention for vectors, while Rust uses `fN` scalar types and explicit vector libraries. The centralized `ASTUtils` class provides a unified `map_primitive_type()` method with per-backend mappings for all nine backends.

Table 5. Representative type mappings across backends

CrossGL	GLSL	HLSL	Metal	CUDA	Rust
<code>vec2</code>	<code>vec2</code>	<code>float2</code>	<code>float2</code>	<code>float2</code>	<code>Vec2</code>
<code>vec4</code>	<code>vec4</code>	<code>float4</code>	<code>float4</code>	<code>float4</code>	<code>Vec4</code>
<code>mat4</code>	<code>mat4</code>	<code>float4x4</code>	<code>float4x4</code>	<code>float4x4</code>	<code>Mat4</code>
<code>ivec3</code>	<code>ivec3</code>	<code>int3</code>	<code>int3</code>	<code>int3</code>	<code>IVec3</code>
<code>sampler2D</code>	<code>sampler2D</code>	<code>Texture2D</code>	<code>texture2d</code>	—	—
<code>float</code>	<code>float</code>	<code>float</code>	<code>float</code>	<code>float</code>	<code>f32</code>
<code>int</code>	<code>int</code>	<code>int</code>	<code>int</code>	<code>int</code>	<code>i32</code>
<code>void</code>	<code>void</code>	<code>void</code>	<code>void</code>	<code>void</code>	<code>()</code>

Semantic mapping. CrossGL maps shader I/O semantics from unified names to platform-specific mechanisms. Table 6 shows representative mappings. GLSL uses built-in variables and layout qualifiers, HLSL uses system-value semantics, and Metal uses C++ attributes.

Table 6. Representative semantic mappings across backends

CrossGL	GLSL	HLSL	Metal
<code>gl.Position</code>	<code>gl.Position</code>	<code>SV_POSITION</code>	<code>[[position]]</code>
<code>gl.FragColor</code>	<code>layout(0)</code>	<code>SV_TARGET</code>	<code>[[color(0)]]</code>
<code>gl.VertexID</code>	<code>gl.VertexID</code>	<code>SV_VertexID</code>	<code>[[vertex_id]]</code>
<code>gl.FragCoord</code>	<code>gl.FragCoord</code>	<code>SV_Position</code>	<code>[[position]]</code>

Function mapping. Built-in shader functions are mapped between naming conventions. For example, CrossGL’s `lerp` maps to GLSL’s `mix` and HLSL’s `lerp`; `frac` maps to GLSL’s `fract` and HLSL’s `frac`; `rsqrt` maps to GLSL’s `inversesqrt`; and `tex2D` maps to GLSL’s `texture`.

5.2 Per-Backend Strategies

While all code generators follow the common traversal pattern described in Section 4.3, each backend addresses platform-specific challenges:

GLSL requires emitting a `#version` directive, translating shader stages to `void main()` entry points, converting struct-based I/O to `layout(location=N)` in/out variable declarations, and mapping CrossGL’s unified semantics to GLSL built-in variables.

HLSL preserves semantic annotations on struct members, generates constant buffer declarations with register bindings, and handles the distinction between texture resources and sampler-state objects.

Metal translates shader entry points to functions with stage attributes such as `vertex`, `fragment`, and `kernel`. It adds device, constant, and threadgroup address-space qualifiers, then restructures parameters around Metal’s argument buffer model.

CUDA and HIP emit kernel functions, map CrossGL global invocation IDs to CUDA/HIP thread and block indices, and translate shared memory declarations to target-specific shared memory variables. HIP generation closely mirrors CUDA, reflecting HIP’s source-level compatibility design.

Rust maps CrossGL types to Rust primitives (`float` $\rightarrow$ `f32`, `int` $\rightarrow$ `i32`, `void` $\rightarrow$ `()`), translates structs to Rust `struct` definitions with `pub` fields, and converts control flow constructs to Rust syntax including expression-based returns.

Mojo translates types to Mojo’s naming conventions (`float` $\rightarrow$ `Float32`, `int` $\rightarrow$ `Int32`, `void` $\rightarrow$ `None`), uses Mojo’s `fn` function syntax, and generates appropriate Python-like indentation-based structure.

5.3 SPIR-V Assembly Generation

The SPIR-V generator targets a structured assembly format rather than a textual source language. It allocates numeric IDs for every type, value, function, and label; deduplicates primitive, vector, matrix, pointer, function, struct, and array types; emits structured control-flow markers such as `OpSelectionMerge` and `OpLoopMerge`; and maps mathematical functions to GLSL.std.450 extended instructions. The backend can call `spirv-as`, `spirv-dis`, and `spirv-val` when those tools are available. This backend remains less mature than the textual backends, as the evaluation in the following section quantifies.

6 Evaluation

We evaluate CrossGL along four dimensions using the reference implementation in the public CrossTL repository at commit `445209d8`: (1) architectural scaling properties, (2) functional translation success across a diverse program corpus, (3) output size and translation latency, and (4) per-backend feature coverage. All experiments were conducted on macOS (Darwin 25.5.0) running on an Apple M2 Max with 32 GB unified memory, using Python 3.14.4.

6.1 Architectural Scalability

The central claim of the hub-and-spoke architecture is that it reduces the engineering effort of multi-language support from quadratic to linear. Table 7 quantifies this advantage.

Table 7. Scaling properties of hub-and-spoke vs. pairwise translation

Languages (N)	Hub-and-spoke ($2N$)	Pairwise ($N(N - 1)$)	Reduction
5	10 converters	20 converters	$2.0\times$
10	20 converters	90 converters	$4.5\times$
15	30 converters	210 converters	$7.0\times$
20	40 converters	380 converters	$9.5\times$

The marginal cost of adding the $(N + 1)$ -th language is constant under the hub-and-spoke model: one importer and one exporter. Pairwise translation requires $2N$ additional converters. This property is independent of any particular implementation and follows directly from the topology shown in Fig. 1.

6.2 Benchmark Corpus and Protocol

The evaluation corpus consists of eight CrossGL programs from five categories, summarized in Table 8. A backend-specific compatibility matrix was used because not every output language is meaningful for every example: graphics shaders were translated to graphics and general-purpose backends, compute kernels to GPU-compute backends, and the PBR shader to the graphics-oriented and Rust backends.

Table 8. Example corpus used for translation evaluation

Category	Program	Purpose	LOC	Targets
Graphics	SimpleShader	Vertex/fragment baseline	42	9
Graphics	PerlinNoise	Math-heavy procedural shader	43	9
Graphics	ComplexShader	PBR-style material shader	499	9
Advanced	ArrayTest	Arrays and composite access	56	9
Advanced	GenericPatternMatching	Generics, enums, match constructs	387	9
Compute	ParticleSimulation	Workgroup compute simulation	287	6
Cross-platform	UniversalPBRShader	Full Cook-Torrance PBR case study	489	5
GPU computing	MatrixMultiplication	CUDA/HIP-style compute kernel	364	4

Each compatible source-target pair was translated five times without external formatting. We report the median wall-clock time per pair. A translation is counted as successful when it completes without exception and produces more than 100 bytes of target output. This criterion tests the CrossGL frontend, universal AST construction, target code generator, and file emission path; it does not claim runtime equivalence of the generated shaders.

6.3 Functional Results

In the evaluated implementation snapshot, the full Python unit suite passed: 767 tests completed in 1.11 s. These tests cover lexers, parsers, backend contracts,

code generation helpers, and native import paths. The end-to-end example harness attempted 60 compatible translations and produced 56 successful target outputs, for a success rate of 93.3%. Across all outputs, CrossGL generated 245,225 bytes and 6,603 lines of target code, with a median translation time of 30.83 ms.

Table 9. End-to-end translation results by backend

Backend	Success	Rate	Median ms	LOC	Observed gaps
CUDA	7/7	100.0%	30.96	651	None in corpus
HLSL	6/7	85.7%	30.44	924	Generic pattern output too small
HIP	7/7	100.0%	30.97	802	None in corpus
Metal	7/7	100.0%	30.32	909	None in corpus
Mojo	6/6	100.0%	23.35	540	None in corpus
GLSL	7/7	100.0%	31.27	895	None in corpus
Rust	7/7	100.0%	42.47	1023	None in corpus
Slang	3/5	60.0%	5.01	85	Complex block traversal; generic output too small
SPIR-V	6/7	85.7%	30.70	774	Enum member handling in generic pattern case

Table 9 shows that the mature textual backends (GLSL, Metal, CUDA, HIP, Rust, and Mojo) handled all compatible examples in the corpus. The remaining failures are concentrated in advanced constructs: Slang generation fails on one complex block traversal and one generic/pattern-matching output, DirectX emits insufficient output for the generic pattern-matching example, and the SPIR-V backend does not yet support enum member lowering for that example.

6.4 Backend Feature Coverage

Table 10. Per-backend feature coverage in the reference implementation

Backend	Export	Import	Best-supported area	Known gaps
GLSL	Full in corpus	Tested	Vertex/fragment graphics with partial compute support	Advanced arrays and generics remain partial
HLSL	Partial	Tested	Graphics semantics and partial compute lowering	Generic pattern matching and enum lowering limited
Metal	Full in corpus	Tested	Vertex/fragment graphics with Metal attributes	Advanced arrays and generics remain partial
SPIR-V	Experimental	Parser tested	Assembly emission for graphics- and compute-shaped inputs	Validation and enum lowering incomplete
CUDA	Full in corpus	Tested	Compute kernels and CUDA-style indexing	Graphics-shaped shaders are syntax-only lowering
HIP	Full in corpus	Tested	Compute kernels and HIP-style indexing	Graphics-shaped shaders are syntax-only lowering
Rust	Full in corpus	Tested	General-purpose output with the strongest advanced-construct coverage	GPU runtime semantics not fully modeled
Mojo	Full in corpus	Tested	General-purpose and compute-shaped output	Advanced arrays and generics remain partial
Slang	Partial	Tested	Basic shader output	Complex block traversal and generic output limited

A coarse count of supported backends does not convey implementation depth. Table 10 therefore separates export/import status from the strongest supported area and the main implementation gaps observed in the evaluated snapshot.

The matrix clarifies two points. First, the architecture supports nine export backends and corresponding native import modules, but backend maturity varies. Second, compute-oriented and general-purpose targets are not semantically equivalent to graphics APIs: CUDA and HIP can receive graphics-shaped programs as syntax, but their strongest support is for compute kernels.

6.5 Comparison with Existing Tools

A direct performance comparison with SPIRV-Cross [9], Tint [10], Naga [11], and DXC [12] is not straightforward because these tools accept different input formats (SPIR-V, WGSL, GLSL, or HLSL respectively) and none accepts CrossGL source. Rewriting the corpus into each tool’s input format would introduce confounds from manual translation quality. We therefore compare along two orthogonal axes: *coverage breadth* and *external interoperability*.

Coverage breadth. Table 2 shows that existing tools support at most six output targets (Slang), while CrossGL reaches nine. More significantly, no existing tool provides bidirectional translation across the full graphics-and-compute spectrum: SPIRV-Cross is unidirectional from SPIR-V; Tint and DXC are single-source; Naga covers four inputs but excludes CUDA, HIP, Rust, and Mojo. CrossGL’s hub-and-spoke model is the only architecture that provides native import paths for GLSL, HLSL, and Metal while simultaneously exporting to all nine targets.

SPIR-V interoperability. We tested the experimental CrossGL SPIR-V assembly outputs with SPIR-V Tools and SPIRV-Cross. Five of seven generated SPIR-V assembly files assembled successfully with `spirv-as`; none passed `spirv-val`, primarily due to incomplete instruction layout and structured module emission. This localizes the main correctness gap: textual source generation is substantially more mature than binary-interchange generation, and closing this gap is a priority for future work.

6.6 Case Study: Cross-Platform PBR Shader

The Universal PBR shader is a 489-line CrossGL program implementing a Cook-Torrance BRDF [21] following the methodology of Pharr et al. [22]. It includes GGX/Trowbridge-Reitz normal distribution, Smith geometry with Schlick-GGX approximation, Schlick Fresnel, cascaded shadow mapping, image-based lighting with prefiltered environment maps, parallax occlusion mapping, and Reinhard tone mapping.

The PBR shader translated successfully to all five compatible targets in the corpus: GLSL (470 LOC), HLSL (444 LOC), Metal (456 LOC), Rust (408 LOC), and SPIR-V assembly (352 LOC). The generated GLSL output includes `#version` directives, layout qualifiers, and uniform blocks; HLSL output uses `SV_` semantic annotations and constant-buffer style declarations; Metal output

uses stage-specific attributes and Metal-style resource types. This case study demonstrates that the CrossGL IR is expressive enough to preserve a substantial physically based lighting pipeline across target syntaxes, while the SPIR-V validation result above shows that interchange-level correctness still requires additional backend work.

6.7 Engineering Effort

The implementation contains 70 Python files and approximately 26,589 non-comment source lines. The reusable translator core accounts for 3,347 LOC, shared forward code generation for 6,622 LOC, and the nine native backend directories for approximately 15,864 LOC. New target support follows a standard template: type mapping, semantic mapping, expression/statement generation, and a `BackendSpec` registration descriptor. This structure is consistent with the theoretical scaling result: backend additions remain localized rather than requiring changes to every existing source-target pair.

7 Discussion

7.1 Limitations

The evaluation exposes three categories of limitations that bound the current system’s applicability.

Uneven backend maturity. GLSL, Metal, CUDA, HIP, Rust, and Mojo handled all compatible corpus examples, while Slang, HLSL, and SPIR-V exhibit gaps on generic pattern matching, enum lowering, or complex block traversal. This unevenness is inherent to the incremental development model: backends mature at different rates depending on the complexity of their target language semantics.

Semantic loss in two-hop translation. Cross-translation between two non-CrossGL languages routes through CrossGL as a pivot, and this intermediate step may discard target-specific idioms. For instance, HLSL’s precise semantics for resource binding slots or Metal’s explicit address-space annotations may be approximated rather than preserved exactly when translating from a third language through the IR. However, any improvement to the CrossGL IR benefits all translation paths simultaneously.

Structural correctness without runtime equivalence. The evaluation measures whether generated code is syntactically well-formed and structurally complete, but does not verify pixel-level equivalence of rendered output or bit-wise identity of compute results. The SPIR-V backend illustrates this boundary: it emits assembly for six of seven compatible examples, but the generated modules do not yet pass `spirv-val`, indicating structural rather than semantic correctness at the binary interchange level.

7.2 Future Work

Several directions follow from the current results, guided by established principles of structured software design for parallel and heterogeneous systems [23].

Runtime validation harness. The highest priority is an automated pipeline that compiles generated GLSL, HLSL, Metal, and SPIR-V through platform toolchains (glslangValidator, fxc/dxc, metal compiler, spirv-val) and compares rendered images or compute buffers within numerical tolerances. This would elevate the evaluation from structural correctness to semantic equivalence.

Optimization passes. The visitor pattern already present in the universal AST facilitates dead code elimination, constant folding, and common subexpression elimination as reusable passes that benefit all backends simultaneously.

Extended target coverage. WGSL support for WebGPU, direct binary SPIR-V and DXIL emission, and extensions for heterogeneous AI accelerators (NPUs, TPUs) would broaden applicability to emerging compute platforms.

Developer tooling. A Language Server Protocol (LSP) implementation for CrossGL would enable syntax highlighting, autocompletion, and error diagnostics in standard editors, lowering the barrier for direct authorship of cross-platform shaders.

8 Conclusion

This paper presented CrossGL, a compiler infrastructure that addresses GPU shading language fragmentation through a hub-and-spoke translation architecture centered on a purpose-built intermediate shader language.

The CrossGL language provides a human-readable, semantically rich IR that captures shader concepts across 16 GPU pipeline stages, including modern ray tracing and mesh shader stages, through a type system supporting generics, pattern matching, and platform-agnostic semantic annotations. The hub-and-spoke architecture reduces the number of required translators from $O(N^2)$ to $O(N)$, with each new language requiring only two converters regardless of how many languages are already supported. In the reference implementation, 767 unit tests passed and 56 of 60 compatible example translations succeeded across nine target backends, including a 489-line cross-platform PBR shader.

The results support the practicality of the architecture while also clarifying its current limits: textual backends are substantially more mature than SPIR-V interchange generation, and advanced constructs such as enums and pattern matching require further backend-specific lowering. The extensible plugin architecture nevertheless gives the system a scalable path forward. As heterogeneous computing architectures proliferate—spanning GPUs, NPUs, TPUs, and specialized AI accelerators—the need for universal translation infrastructure will only grow. The hub-and-spoke model provides a principled foundation for bridging these platforms through a single, shared intermediate representation.

Acknowledgments. The authors thank the anonymous reviewers for their constructive feedback, which improved the evaluation methodology and presentation of this work.

Declaration on Generative AI. Generative AI assistance was used during camera-ready preparation for language refinement and proofreading. The authors reviewed, edited, and take full responsibility for all technical claims, results, and final manuscript content.

References

1. Kessenich, J., Sellers, G., Shreiner, D.: OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 4.5 with SPIR-V, 9th edn. Addison-Wesley, Boston (2016)
2. Microsoft Corporation: HLSL Programming Guide. Microsoft Learn (2023). <https://learn.microsoft.com/en-us/windows/win32/direct3dhls/>
3. Apple Inc.: Metal Shading Language Specification, Version 3.1. Apple Inc. (2023). <https://developer.apple.com/metal/Metal-Shading-Language-Specification.pdf>
4. Khronos Group: SPIR-V Specification, Version 1.6, Revision 2. Khronos Group (2022). <https://registry.khronos.org/SPIR-V/>
5. Nickolls, J., Buck, I., Garland, M., Skadron, K.: Scalable Parallel Programming with CUDA. ACM Queue **6**(2), 40–53 (2008). <https://doi.org/10.1145/1365490.1365500>
6. Advanced Micro Devices, Inc.: HIP Programming Guide. AMD ROCm Documentation (2023). <https://rocm.docs.amd.com/projects/HIP/>
7. Modular Inc.: The Mojo Programming Language. Modular (2023). <https://www.modular.com/mojo>
8. He, Y., Fatahalian, K., Foley, T.: Slang: Language Mechanisms for Extensible Real-Time Shading Systems. ACM Trans. Graph. **37**(4), Article 1 (2018). <https://doi.org/10.1145/3197517.3201380>
9. Khronos Group: SPIRV-Cross – A Practical Tool for SPIR-V Reflection and Disassembly. GitHub Repository (2023). <https://github.com/KhronosGroup/SPIRV-Cross>
10. Google: Tint – A Compiler for the WebGPU Shading Language (WGSL). Part of the Dawn project (2023). <https://dawn.googlesource.com/tint>
11. Mozilla: Naga – Universal Shader Translation in Rust. Part of the wgpu project (2023). <https://github.com/gfx-rs/wgpu/tree/trunk/naga>
12. Microsoft Corporation: DirectX Shader Compiler (DXC). GitHub Repository (2023). <https://github.com/microsoft/DirectXShaderCompiler>
13. Embark Studios: rust-gpu – Making Rust a First-Class Language and Ecosystem for GPU Shaders. GitHub Repository (2023). <https://github.com/EmbarkStudios/rust-gpu>
14. Owens, J.D., Houston, M., Luebke, D., Green, S., Stone, J.E., Phillips, J.C.: GPU Computing. Proceedings of the IEEE **96**(5), 879–899 (2008). <https://doi.org/10.1109/JPROC.2008.917757>
15. Aho, A.V., Lam, M.S., Sethi, R., Ullman, J.D.: Compilers: Principles, Techniques, and Tools, 2nd edn. Pearson/Addison-Wesley, Boston (2006)

16. Lattner, C., Adve, V.: LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In: Proceedings of the International Symposium on Code Generation and Optimization (CGO), pp. 75–86. IEEE Computer Society (2004). <https://doi.org/10.1109/CGO.2004.1281665>
17. Lattner, C., Amini, M., Bondhugula, U., Cohen, A., Davis, A., Pienaar, J., Riddle, R., Shpeisman, T., Vasilache, N., Zinenko, O.: MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. In: Proceedings of the International Symposium on Code Generation and Optimization (CGO), pp. 2–14. IEEE (2021). <https://doi.org/10.1109/CGO51591.2021.9370308>
18. Mark, W.R., Glanville, R.S., Akeley, K., Kilgard, M.J.: Cg: A System for Programming Graphics Hardware in a C-like Language. *ACM Trans. Graph.* **22**(3), 896–907 (2003). <https://doi.org/10.1145/882262.882362>
19. Sellers, G., Kessenich, J.: Vulkan Programming Guide: The Official Guide to Learning Vulkan. Addison-Wesley, Boston (2017)
20. Kessenich, J., Baldwin, D., Rost, R.: The OpenGL Shading Language, Version 4.60. Khronos Group (2017). <https://registry.khronos.org/OpenGL/specs/gl/GLSLangSpec.4.60.pdf>
21. Cook, R.L., Torrance, K.E.: A Reflectance Model for Computer Graphics. *ACM Trans. Graph.* **1**(1), 7–24 (1982). <https://doi.org/10.1145/357290.357293>
22. Pharr, M., Jakob, W., Humphreys, G.: Physically Based Rendering: From Theory to Implementation, 4th edn. MIT Press, Cambridge (2023)
23. McCool, M., Reinders, J., Robison, A.: Structured Parallel Programming: Patterns for Efficient Computation. Morgan Kaufmann, San Francisco (2012)